

7 Mapping inheritance

This chapter covers

- Examining inheritance-mapping strategies
- Investigating polymorphic associations

We deliberately haven't talked much about inheritance mapping so far. Mapping makes the connection between the object-oriented world and the relational world, but inheritance is specific to object-oriented systems. Consequently, mapping a hierarchy of classes to tables can be a complex problem, and we'll demonstrate various strategies in this chapter.

A basic strategy for mapping classes to database tables might be “one table for every persistent entity class.” This approach sounds simple enough and indeed works well until we encounter inheritance.

Inheritance is such a visible structural mismatch between the object-oriented and relational worlds because the object-oriented model provides both *is a* and *has a* relationships. SQL-based models provide only *has a* relationships; SQL database management systems don't support type inheritance, and even when it's available, it's usually proprietary or incomplete.

There are four different strategies for representing an inheritance hierarchy:

- Use one table per concrete class and default runtime polymorphic behavior.
- Use one table per concrete class, but discard polymorphism and inheritance relationships completely from the SQL schema. Use SQL `UNION` queries for runtime polymorphic behavior.
- Use one table per class hierarchy: enable polymorphism by denormalizing the SQL schema and rely on row-based discrimination to determine supertypes and subtypes.
- Use one table per subclass: represent *is a* (inheritance) relationships as *has a* (foreign key) relationships, and use SQL `JOIN` operations.

This chapter takes a top-down approach, assuming that we're starting with a domain model and trying to derive a new SQL schema. The map-

ping strategies described here are just as relevant if you're working from the bottom up, starting with existing database tables. We'll examine some tricks along the way to help you deal with imperfect table layouts.

7.1 Table per concrete class with implicit polymorphism

We are working on the CaveatEmptor application, implementing persistence for a hierarchy of classes. We could stick with the simplest approach suggested: use exactly one table for each concrete class. We can map all the properties of a class, including inherited properties, to columns of a table, as shown in figure 7.1.

NOTE To be able to execute the examples from this chapter's source code, you will first need to run the Ch07.sql script.

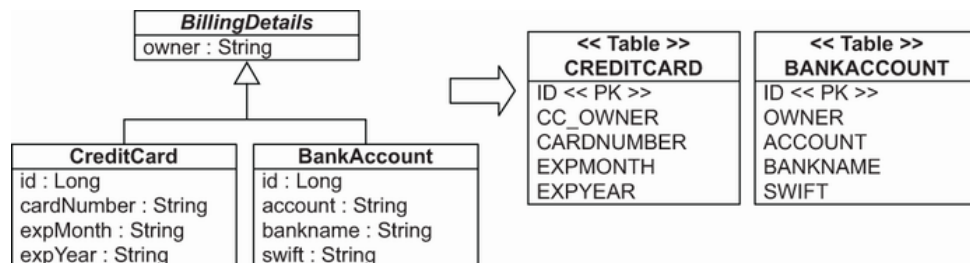


Figure 7.1 Mapping all concrete classes to an independent table

Relying on this implicit polymorphism, we'll map concrete classes with `@Entity`, as usual. By default, properties of the superclass are ignored and not persistent! We'll have to annotate the superclass with `@MappedSuperclass` to enable the embedding of its properties in the concrete subclass tables; see listing 7.1, which can be found in the `mapping-inheritance-mappedsuperclass` folder.

Listing 7.1 Mapping `BillingDetails` (abstract superclass) with implicit polymorphism

```
Path: Ch07/mapping-inheritance-mappedsuperclass/src/main/java/com/manni
➡ /javapersistence/ch07/model/BillingDetails.java
```

```
@MappedSuperclass
\1 BillingDetails {
    @Id
    @GeneratedValue(generator = "ID_GENERATOR")
    private Long id;

    @NotNull
    private String owner;
    // . . .
}
```

Now we'll map the concrete subclasses.

Listing 7.2 Mapping `CreditCard` (concrete subclass)

```
Path: Ch07/mapping-inheritance-mappedsuperclass/src/main/java/com/manni
➡ /javapersistence/ch07/model/CreditCard.java

@Entity
@AttributeOverride(
    name = "owner",
    column = @Column(name = "CC_OWNER", nullable = false))
public class CreditCard extends BillingDetails {

    @NotNull
    private String cardNumber;

    @NotNull
    private String expMonth;

    @NotNull
    private String expYear;

    // . . .
}
```

We can override column mappings from the superclass in a subclass with the `@AttributeOverride` annotation. Starting with JPA 2.2, we can use several `@AttributeOverride` annotations on the same class; up to JPA 2.1, we had to group the `@AttributeOverride` annotations within an `@AttributeOverrides` annotation. The previous example renames the `OWNER` column to `CC_OWNER` in the `CREDITCARD` table.

The following listing shows the mapping of the `BankAccount` subclass.

Listing 7.3 Mapping `BankAccount` (concrete subclass)

```
Path: Ch07/mapping-inheritance-mappedsuperclass/src/main/java/com/manni
➡ /javapersistence/ch07/model/BankAccount.java

@Entity
public class BankAccount extends BillingDetails {

    @NotNull
    private String account;

    @NotNull
    private String bankname;

    @NotNull
    private String swift;
}
```

```
// . . .
}
```

We can declare the identifier property in the superclass, with a shared column name and generator strategy for all subclasses (as in listing 7.3), or we can repeat it inside each concrete class.

To work with these classes, we'll create three Spring Data JPA repository interfaces.

Listing 7.4 The `BillingDetailsRepository` interface

```
Path: Ch07/mapping-inheritance-mappedsuperclass/src/main/java/com/manni
➡ /javapersistence/ch07/repositories/BillingDetailsRepository.java

@NoRepositoryBean
public interface BillingDetailsRepository<T extends BillingDetails, ID>
    extends JpaRepository<T, ID> {
    List<T> findByOwner(String owner);
}
```

In the preceding listing, the `BillingDetailsRepository` interface is annotated with `@NoRepositoryBean`. This prevents its instantiation as a Spring Data JPA repository instance. This is necessary because, following the schema from figure 7.1, there will be no `BILLINGDETAILS` table. However, the `BillingDetailsRepository` interface intends to be extended by the repository interfaces to deal with the `CreditCard` and `BankAccount` subclasses. That is why `BillingDetailsRepository` is generified by a `T` that extends `BillingDetails`. Additionally, it contains the `findByOwner` method. The `owner` field from `BillingDetails` will be included in both the `CREDITCARD` and `BANKACCOUNT` tables.

We'll now create two more Spring Data repository interfaces.

Listing 7.5 The `BankAccountRepository` interface

```
Path: Ch07/mapping-inheritance-mappedsuperclass/src/main/java/com/manni
➡ /javapersistence/ch07/repositories/BankAccountRepository.java

public interface BankAccountRepository
    extends BillingDetailsRepository<BankAccount, Long> {
    List<BankAccount> findBySwift(String swift);
}
```

The `BankAccountRepository` interface extends `BillingDetailsRepository`, generified by `BankAccount` (as it deals with `BankAccount` instances) and by `Long` (as the ID of the class is of this type). It adds the `findBySwift` method, whose name follows the Spring Data JPA conventions (see chapter 4).

Listing 7.6 The `CreditCardRepository` interface

Path: Ch07/mapping-inheritance-mappedsuperclass/src/main/java/com/manni
➡ /javapersistence/ch07/repositories/CreditCardRepository.java

```
public interface CreditCardRepository
    extends BillingDetailsRepository<CreditCard, Long> {
    List<CreditCard> findByExpYear(String expYear);
}
```

The `CreditCardRepository` interface extends `BillingDetailsRepository`, generified by `CreditCard` (as it deals with `CreditCard` instances) and by `Long` (as the ID of the class is of this type). It adds the `findByExpYear` method, whose name follows the Spring Data JPA conventions (see chapter 4).

We'll create the following test to check the functionality of the persistence code.

Listing 7.7 Testing the functionality of the persistence code

Path: Ch07/mapping-inheritance-mappedsuperclass/src/test/java/com/manni
➡ /javapersistence/ch07/MappingInheritanceSpringDataJPATest.java

```
@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = {SpringDataConfiguration.class})
public class MappingInheritanceSpringDataJPATest {

    @Autowired
    private CreditCardRepository creditCardRepository;

    @Autowired
    private BankAccountRepository bankAccountRepository;

    @Test
    void storeLoadEntities() {

        CreditCard creditCard = new CreditCard(
            "John Smith", "123456789", "10", "2030");
        creditCardRepository.save(creditCard);

        BankAccount bankAccount = new BankAccount(
            "Mike Johnson", "12345", "Delta Bank", "BANKXY12");
```

```

        bankAccountRepository.save(bankAccount);

        List<CreditCard> creditCards =
            creditCardRepository.findByOwner("John Smith");
        List<BankAccount> bankAccounts =
            bankAccountRepository.findByOwner("Mike Johnson");
        List<CreditCard> creditCards2 =
            creditCardRepository.findByExpYear("2030");
        List<BankAccount> bankAccounts2 =
            bankAccountRepository.findBySwift("BANKXY12");

        assertAll(
            () -> assertEquals(1, creditCards.size()),
            () -> assertEquals("123456789",
                creditCards.get(0).getCardNumber()),
            () -> assertEquals(1, bankAccounts.size()),
            () -> assertEquals("12345",
                bankAccounts.get(0).getAccount()),
            () -> assertEquals(1, creditCards2.size()),
            () -> assertEquals("John Smith",
                creditCards2.get(0).getOwner()),
            () -> assertEquals(1, bankAccounts2.size()),
            () -> assertEquals("Mike Johnson",
                bankAccounts2.get(0).getOwner())
        );
    }
}

```

Ⓐ Extend the test using `SpringExtension`. This extension is used to integrate the Spring test context with the JUnit 5 Jupiter test.

Ⓑ The Spring test context is configured using the beans defined in the `SpringDataConfiguration` class.

Ⓒ A `CreditCardRepository` bean is injected by Spring through autowiring.

Ⓓ A `BankAccountRepository` bean is injected by Spring through autowiring. This is possible as the `com.manning.javapersistence.ch07.repositories` package where `CreditCardRepository` and `BankAccountRepository` are located was used as the argument of the `@EnableJpaRepositories` annotation on the `SpringDataConfiguration` class. To recall what the `SpringDataConfiguration` class looks like, refer to chapter 2.

Ⓔ Create a credit card and save it to the repository.

Ⓕ Create a bank account and save it to the repository.

- Ⓒ Get the list of all credit cards having John Smith as owner.
- Ⓓ Get the list of all bank accounts having Mike Johnson as owner.
- Ⓔ Get the credit cards expiring in 2030.
- Ⓕ Get the bank accounts with SWIFT BANKXY12.
- Ⓖ Check the size of the list of credit cards.
- Ⓗ Get the number of the first credit card in the list.
- Ⓜ Check the size of the list of bank accounts.
- Ⓝ Check the number of the first bank account in the list.
- Ⓞ Check the size of the list of credit cards expiring in 2030.
- Ⓟ Check the owner of the first credit card in this list.
- Ⓠ Check the size of the list of bank accounts with SWIFT BANKXY12.
- Ⓡ Check the owner of the first bank account in this list.

The source code for this chapter also demonstrates how to test these classes using JPA and Hibernate.

The main problem with implicit inheritance mapping is that it doesn't support polymorphic associations very well. In the database, we usually represent associations as foreign key relationships. In the schema from figure 7.1, if the subclasses are all mapped to different tables, a polymorphic association to their superclass (the abstract `BillingDetails`) can't be represented as a simple foreign key relationship. We can't have another entity mapped with a foreign key "referencing `BILLINGDETAILS`"—there is no such table. This would be problematic in the domain model because `BillingDetails` is associated with `User`; both the `CREDITCARD` and `BANKACCOUNT` tables would need a foreign key reference to the `USERS` table. None of these problems can be easily resolved, so we should consider an alternative mapping strategy.

Polymorphic queries that return instances of all classes that match the interface of the queried class are also problematic. Hibernate must execute a query against the superclass as several SQL `SELECT`s—one for each concrete subclass. The JPA query `select bd from BillingDetails bd` requires two SQL statements:

```
select
    ID, OWNER, ACCOUNT, BANKNAME, SWIFT
```

```

from
    BANKACCOUNT
select
    ID, CC_OWNER, CARDNUMBER, EXPMONTH, EXPYEAR
from
    CREDITCARD

```

Hibernate or Spring Data JPA using Hibernate uses a separate SQL query for each concrete subclass. On the other hand, queries against the concrete classes are trivial and perform well—Hibernate uses only one of the statements.

A further conceptual problem with this mapping strategy is that several different columns of different tables share exactly the same semantics. This makes schema evolution more complex. For example, renaming or changing the type of a superclass property results in changes to multiple columns in multiple tables. Many of the standard refactoring operations offered by your IDE would require manual adjustments, because the automatic procedures usually don't count for things like `@AttributeOverride` or `@AttributeOverrides`. It is much more difficult to implement database integrity constraints that apply to all subclasses.

We recommend this approach only for the top level of your class hierarchy, where polymorphism isn't usually required and when modification of the superclass in the future is unlikely. This may work for particular domain models that you face in your real-life applications, but it isn't a good fit for the CaveatEmptor domain model, where queries and other entities refer to `BillingDetails`. We'll look for other alternatives.

With the help of the SQL `UNION` operation, we can eliminate most of the concerns with polymorphic queries and associations.

7.2 Table per concrete class with unions

Let's consider a union subclass mapping with `BillingDetails` as an abstract class (or interface), as in the previous section. In this situation, there are again two tables and superclass columns that are duplicated in both: `CREDITCARD` and `BANKACCOUNT`. What's new here is an inheritance strategy known as `TABLE_PER_CLASS`, declared on the superclass, as shown in the following listing. The source code can be found in the `mapping-inheritance-tableperclass` folder.

NOTE The JPA standard specifies that `TABLE_PER_CLASS` is optional, so not all JPA implementations support it.

Listing 7.8 Mapping `BillingDetails` with `TABLE_PER_CLASS`


```
Path: Ch07/mapping-inheritance-tableperclass/src/main/java/com/manning
➡ /javapersistence/ch07/model/BillingDetails.java
```

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class BillingDetails {
    @Id
    @GeneratedValue(generator = "ID_GENERATOR")
    private Long id;
    @NotNull
    private String owner;
    // . . .
}
```

The database identifier and its mapping have to be present in the superclass to share it in all subclasses and their tables. This is no longer optional, as it was for the previous mapping strategy. The `CREDITCARD` and `BANKACCOUNT` tables both have an `ID` primary key column. All concrete class mappings inherit persistent properties from the superclass (or interface). An `@Entity` annotation on each subclass is all that is required.

Listing 7.9 Mapping `CreditCard`

```
Path: Ch07/mapping-inheritance-tableperclass/src/main/java/com/manning
➡ /javapersistence/ch07/model/CreditCard.java
```

```
@Entity
@AttributeOverride(
    name = "owner",
    column = @Column(name = "CC_OWNER", nullable = false))
public class CreditCard extends BillingDetails {
    @NotNull
    private String cardNumber;
    @NotNull
    private String expMonth;
    @NotNull
    private String expYear;
    // . . .
}
```

Listing 7.10 Mapping `BankAccount`

```
Path: Ch07/mapping-inheritance-tableperclass/src/main/java/com/manning
➡ /javapersistence/ch07/model/BankAccount.java
```

```
@Entity
public class BankAccount extends BillingDetails {
    @NotNull
    private String account;
```

```

        @NotNull
        private String bankName;
        @NotNull
        private String swift;
        // . . .
    }

```

We'll have to change the `BillingDetailsRepository` interface and remove the `@NoRepositoryBean` annotation. This change, together with the fact that the `BillingDetails` class is now annotated as `@Entity`, will allow this repository to interact with the database. This is what the `BillingDetailsRepository` interface looks like now.

Listing 7.11 The `BillingDetailsRepository` interface

```

Path: Ch07/mapping-inheritance-tableperclass/src/main/java/com/manning
➡ /javapersistence/ch07/model/BillingDetailsRepository.java

public interface BillingDetailsRepository<T extends BillingDetails, ID>
    extends JpaRepository<T, ID> {
    List<T> findByOwner(String owner);
}

```

Keep in mind that the SQL schema still isn't aware of the inheritance; the tables look exactly alike, as shown in figure 7.1.

If `BillingDetails` were concrete, we'd need an additional table to hold instances. Keep in mind that there is still no relationship between the database tables, except for the fact that they have some (many) similar columns.

The advantages of this mapping strategy are clearer if we examine polymorphic queries.

We can use the Spring Data JPA `BillingDetailsRepository` interface to query the database, like this:

```

billingDetailsRepository.findAll();

```

Or from JPA or Hibernate, we can execute the following query:

```

select bd from BillingDetails bd

```

Both approaches will generate the following SQL statement:

```

select
    ID, OWNER, EXPMONTH, EXPYEAR, CARDNUMBER,
    ACCOUNT, BANKNAME, SWIFT, CLAZZ_
from
    ( select
        ID, OWNER, EXPMONTH, EXPYEAR, CARDNUMBER,
        null as ACCOUNT,
        null as BANKNAME,
        null as SWIFT,
        1 as CLAZZ_
    from
        CREDITCARD
    union all
    select
        id, OWNER,
        null as EXPMONTH,
        null as EXPYEAR,
        null as CARDNUMBER,
        ACCOUNT, BANKNAME, SWIFT,
        2 as CLAZZ_
    from
        BANKACCOUNT
    ) as BILLINGDETAILS

```

This `SELECT` uses a `FROM`-clause subquery to retrieve all instances of `BillingDetails` from all concrete class tables. The tables are combined with a `UNION` operator, and a literal (in this case, `1` and `2`) is inserted into the intermediate result; Hibernate reads this to instantiate the correct class given the data from a particular row. A union requires that the queries that are combined project over the same columns, so you have to pad and fill nonexistent columns with `NULL`. You may wonder whether this query will really perform better than two separate statements. Here you can let the database optimizer find the best execution plan to combine rows from several tables instead of merging two result sets in memory as Hibernate's polymorphic loader engine would do.

An important advantage is the ability to handle polymorphic associations; for example, an association mapping from `User` to `BillingDetails` will now be possible. Hibernate can use a `UNION` query to simulate a single table as the target of the association mapping.

So far, the inheritance-mapping strategies we've examined don't require extra consideration of the SQL schema. This situation changes with the next strategy.

7.3 Table per class hierarchy

We could map an entire class hierarchy to a single table. This table includes columns for all properties of all classes in the hierarchy. The value of an extra type discriminator column or formula identifies the concrete subclass represented by a particular row. Figure 7.2 shows this approach. The source code that follows can be found in the `mapping-inheritance-singletable` folder.

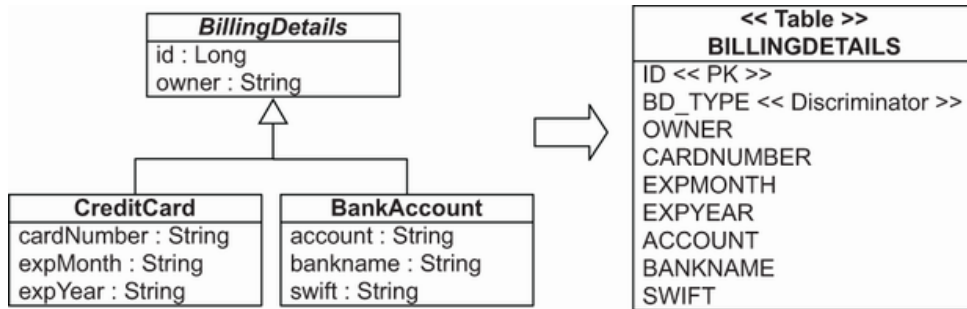


Figure 7.2 Mapping an entire class hierarchy to a single table

This mapping strategy is a winner in terms of both performance and simplicity. It's the best-performing way to represent polymorphism—both polymorphic and non-polymorphic queries perform well, and it's even easy to write queries by hand. Ad hoc reporting is possible without complex joins or unions. Schema evolution is straightforward.

There is one major problem: data integrity. We must declare columns for properties declared by subclasses to be nullable. If the subclasses each define several non-nullable properties, the loss of `NOT NULL` constraints may be a serious problem from the point of view of data correctness. Imagine that an expiration date for credit cards is required, but the database schema can't enforce this rule because all columns of the table can be `NULL`. A simple application programming error can lead to invalid data.

Another important concern is normalization. We've created functional dependencies between non-key columns, violating the third normal form. As always, denormalization for performance reasons can be misleading because it sacrifices long-term stability, maintainability, and the integrity of data for immediate gains that may also be achieved by proper optimization of the SQL execution plans (in other words, ask the DBA).

We'll use the `SINGLE_TABLE` inheritance strategy to create a table-per-class hierarchy mapping, as shown in the following listing.

Listing 7.12 Mapping `BillingDetails` with `SINGLE_TABLE`

```
Path: Ch07/mapping-inheritance-singletable/src/main/java/com/manning
➡ /javapersistence/ch07/model/BillingDetails.java
```

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "BD_TYPE")
public abstract class BillingDetails {

    @Id
    @GeneratedValue(generator = "ID_GENERATOR")
    private Long id;

    @NotNull
    @Column(nullable = false)
    private String owner;

    // . . .
}
```

The root class of the inheritance hierarchy, `BillingDetails`, is mapped to the `BILLINGDETAILS` table automatically. Shared properties of the superclass can be `NOT NULL` in the schema; every subclass instance must have a value. An implementation quirk of Hibernate requires that we declare nullability with `@Column` because Hibernate ignores Bean Validation's `@NotNull` when it generates the database schema.

We have to add a special discriminator column to distinguish what each row represents. This isn't a property of the entity; it's used internally by Hibernate. The column name is `BD_TYPE`, and the values are strings—in this case, `"CC"` or `"BA"`. Hibernate or Spring Data JPA using Hibernate automatically sets and retrieves the discriminator values.

If we don't specify a discriminator column in the superclass, its name defaults to `DTYPE`, and the values are strings. All concrete classes in the inheritance hierarchy can have a discriminator value, such as `CreditCard`.

Listing 7.13 Mapping `CreditCard` using the `SINGLE_TABLE` inheritance strategy

```
Path: Ch07/mapping-inheritance-singletable/src/main/java/com/manning
➡ /javapersistence/ch07/model/CreditCard.java
```

```
@Entity
@DiscriminatorValue("CC")
public class CreditCard extends BillingDetails {
    @NotNull
    private String cardNumber;
    @NotNull
```

```

        private String expMonth;
        @NotNull
        private String expYear;
        // . . .
    }

```

Without an explicit discriminator value, Hibernate defaults to the fully qualified class name if we use Hibernate XML files and to the simple entity name if we use annotations or JPA XML files. Note that JPA doesn't specify a default for non-string discriminator types; each persistence provider can have different defaults. Therefore, we should always specify discriminator values for the concrete classes.

We'll annotate every subclass with `@Entity`, and then map properties of a subclass to columns in the `BILLINGDETAILS` table. Remember that `NOT NULL` constraints aren't allowed in the schema because a `BankAccount` instance won't have an `expMonth` property, and the `EXPMONTH` column must be `NULL` for that row. Hibernate and Spring Data JPA using Hibernate ignore the `@NotNull` for schema DDL generation, but they observe it at runtime before inserting a row. This helps us avoid programming errors; we don't want to accidentally save credit card data without its expiration date. (Other, less well-behaved applications can, of course, still store incorrect data in this database.)

We can use the Spring Data JPA `BillingDetailsRepository` interface to query the database, like this:

```

billingDetailsRepository.findAll();

```

Or, from JPA or Hibernate, we can execute the following query:

```

select bd from BillingDetails bd

```

Both approaches will generate the following SQL statement:

```

select
    ID, OWNER, EXPMONTH, EXPYEAR, CARDNUMBER,
    ACCOUNT, BANKNAME, SWIFT, BD_TYPE
from
    BILLINGDETAILS

```

To query the `CreditCard` subclass, we also have alternatives.

We can use the Spring Data JPA `CreditCardRepository` interface to query the database, like this:

```
creditCardRepository.findAll();
```

Or, from JPA or Hibernate, we can execute the following query:

```
select cc from CreditCard cc
```

Hibernate adds a restriction on the discriminator column:

```
select
    ID, OWNER, EXPMONTH, EXPYEAR, CARDNUMBER
from
    BILLINGDETAILS
where
    BD_TYPE='CC'
```

Sometimes, especially in legacy schemas, we don't have the freedom to include an extra discriminator column in the entity tables. In this case, we can apply an expression to calculate a discriminator value for each row. Formulas for discrimination aren't part of the JPA specification, but Hibernate has an extension annotation, `@DiscriminatorFormula`.

Listing 7.14 Mapping `BillingDetails` with a `@DiscriminatorFormula`

```
Path: Ch07/mapping-inheritance-singletableformula/src/main/java/com/mar
➡ /javapersistence/ch07/model/BillingDetails.java
```

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@org.hibernate.annotations.DiscriminatorFormula(
    "case when CARDNUMBER is not null then 'CC' else 'BA' end"
)
public abstract class BillingDetails {
    // . . .
}
```

There is no discriminator column in the schema, so this mapping relies on an SQL `CASE/WHEN` expression to determine whether a particular row represents a credit card or a bank account (many developers have never used this kind of SQL expression; check the ANSI standard if you aren't familiar with it). The result of the expression is a literal, `CC` or `BA`, declared on the subclass mappings.

The disadvantages of the table-per-class hierarchy strategy may be too serious for your design—denormalized schemas can become a major bur-

den in the long term, and your DBA may not like it at all. The next inheritance-mapping strategy doesn't expose you to this problem.

7.4 Table per subclass with joins

The fourth option is to represent inheritance relationships as SQL foreign key associations. Every class or subclass that declares persistent properties—including abstract classes and even interfaces—has its own table. The source code that follows can be found in the mapping-inheritance-joined folder.

Unlike the table-per-concrete-class strategy we mapped first, the table of a concrete `@Entity` here contains columns only for each non-inherited property declared by the subclass itself, along with a primary key that is also a foreign key of the superclass table. This is easier than it sounds; have a look at figure 7.3.

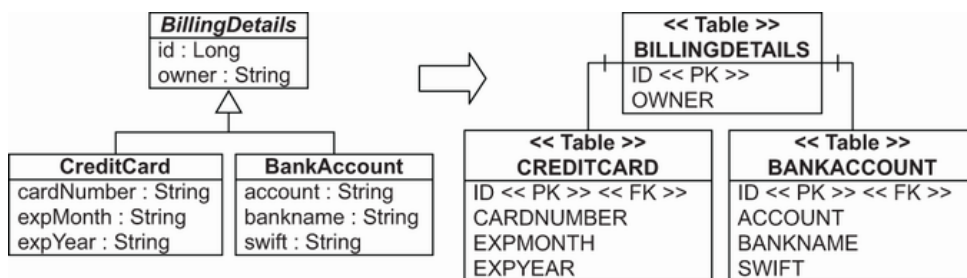


Figure 7.3 Mapping all classes of the hierarchy to their own tables

If we make an instance of the `CreditCard` subclass persistent, Hibernate inserts two rows: The values of properties declared by the `BillingDetails` superclass are stored in a new row of the `BILLINGDETAILS` table. Only the values of properties declared by the subclass are stored in a new row of the `CREDITCARD` table. The primary key shared by the two rows links them together. Later, the subclass instance can be retrieved from the database by joining the subclass table with the superclass table.

The primary advantage of this strategy is that it normalizes the SQL schema. Schema evolution and integrity-constraint definition are straightforward. A foreign key referencing the table of a particular subclass may represent a polymorphic association to that particular subclass. We'll use the `JOINED` inheritance strategy to create a table-per-subclass hierarchy mapping.

Listing 7.15 Mapping `BillingDetails` with `JOINED`

```
Path: Ch07/mapping-inheritance-joined/src/main/java/com/manning
➡ /javapersistence/ch07/model/BillingDetails.java
```



```

@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class BillingDetails {
    @Id
    @GeneratedValue(generator = "ID_GENERATOR")
    private Long id;
    @NotNull
    private String owner;

    // . . .
}

```

The root class `BillingDetails` is mapped to the table `BILLINGDETAILS`. Note that no discriminator is required with this strategy.

In subclasses, we don't need to specify the join column if the primary key column of the subclass table has (or is supposed to have) the same name as the primary key column of the superclass table. In the following listing, `BankAccount` will be a subclass of `BillingDetails`.

Listing 7.16 Mapping `BankAccount` (concrete class)

Path: Ch07/mapping-inheritance-joined/src/main/java/com/manning
 ➤ /javapersistence/ch07/model/BankAccount.java

```

@Entity
public class BankAccount extends BillingDetails {

    @NotNull
    private String account;

    @NotNull
    private String bankname;

    @NotNull
    private String swift;

    // . . .
}

```

This entity has no identifier property; it automatically inherits the `ID` property and column from the superclass, and Hibernate knows how to join the tables if we want to retrieve instances of `BankAccount`.

Of course, we could specify the column name explicitly, using the `@PrimaryKeyJoinColumn` annotation, as shown in the following listing.

Listing 7.17 Mapping `CreditCard`

```
Path: Ch07/mapping-inheritance-joined/src/main/java/com/manning
➡ /javapersistence/ch07/model/CreditCard.java
```

```
@Entity
@PrimaryKeyJoinColumn(name = "CREDITCARD_ID")
public class CreditCard extends BillingDetails {
    @NotNull
    private String cardNumber;
    @NotNull
    private String expMonth;
    @NotNull
    private String expYear;
    // . . .
}
```

The primary key columns of the `BANKACCOUNT` and `CREDITCARD` tables each also have a foreign key constraint referencing the primary key of the `BILLINGDETAILS` table.

We can use the Spring Data JPA `BillingDetailsRepository` interface to query the database, like this:

```
billingDetailsRepository.findAll();
```

Or, from JPA or Hibernate, we can execute the following query:

```
select bd from BillingDetails bd
```

Hibernate relies on an SQL outer join and will generate the following:

```
select
    BD.ID, BD.OWNER,
    CC.EXPMONTH, CC.EXPYEAR, CC.CARDNUMBER,
    BA.ACCOUNT, BA.BANKNAME, BA.SWIFT,
    case
        when CC.CREDITCARD_ID is not null then 1
        when BA.ID is not null then 2
        when BD.ID is not null then 0
    end
from
    BILLINGDETAILS BD
    left outer join CREDITCARD CC on BD.ID=CC.CREDITCARD_ID
    left outer join BANKACCOUNT BA on BD.ID=BA.ID
```

The SQL `CASE . . . WHEN` clause detects the existence (or absence) of rows in the subclass tables `CREDITCARD` and `BANKACCOUNT`, so

Hibernate or Spring Data using Hibernate can determine the concrete subclass for a particular row of the `BILLINGDETAILS` table.

For a narrow subclass query like this,

```
creditCardRepository.findAll();
```

or this,

```
select cc from CreditCard cc,
```

Hibernate uses an inner join:

```
select
    CREDITCARD_ID, OWNER, EXPMONTH, EXPYEAR, CARDNUMBER
from
    CREDITCARD
    inner join BILLINGDETAILS on CREDITCARD_ID=ID
```

As you can see, this mapping strategy is more difficult to implement by hand—even ad hoc reporting is more complex. This is an important consideration if you plan to mix Spring Data JPA or Hibernate code with handwritten SQL. A usual approach and a portable solution may be working with JPQL (Jakarta Persistence Query Language) and annotating methods with JPQL queries.

Furthermore, even though this mapping strategy is deceptively simple, our experience is that performance can be unacceptable for complex class hierarchies. Queries always require a join across many tables or many sequential reads.

Inheritance with joins and a discriminator

Hibernate doesn't need a special discriminator database column to implement the `InheritanceType.JOINED` strategy, and the JPA specification doesn't contain any requirements either. The `CASE . . . WHEN` clause in the SQL `SELECT` statement is a smart way to distinguish the entity type of each retrieved row.

Some JPA examples you might find elsewhere, however, use

`InheritanceType.JOINED` and a `@DiscriminatorColumn` mapping.

Apparently some other JPA providers don't use `CASE . . . WHEN` clauses and rely only on a discriminator value, even for the

`InheritanceType.JOINED` strategy. Hibernate doesn't need the discriminator but uses a declared `@DiscriminatorColumn`, even with a

JOINED mapping strategy. If you prefer to ignore the discriminator mapping with JOINED (it was ignored in older Hibernate versions), enable the configuration property `hibernate.discriminator.ignore_explicit_for_joined`.

Before we look at when you should choose which strategy, let's consider mixing inheritance-mapping strategies in a single class hierarchy.

7.5 Mixing inheritance strategies

We can map an entire inheritance hierarchy with the

`TABLE_PER_CLASS`, `SINGLE_TABLE`, or `JOINED` strategy. We can't mix them—for example, switching from a table-per-class hierarchy with a discriminator to a normalized table-per-subclass strategy. Once we've decided on an inheritance strategy, we have to stick with it.

Except, this isn't completely true. By using some tricks, we can switch the mapping strategy for a particular subclass. For example, we can map a class hierarchy to a single table, but, for a particular subclass, switch to a separate table with a foreign key-mapping strategy, just as with table-per-subclass. Look at the schema in figure 7.4. The source code that follows can be found in the `mapping-inheritance-mixed` folder.

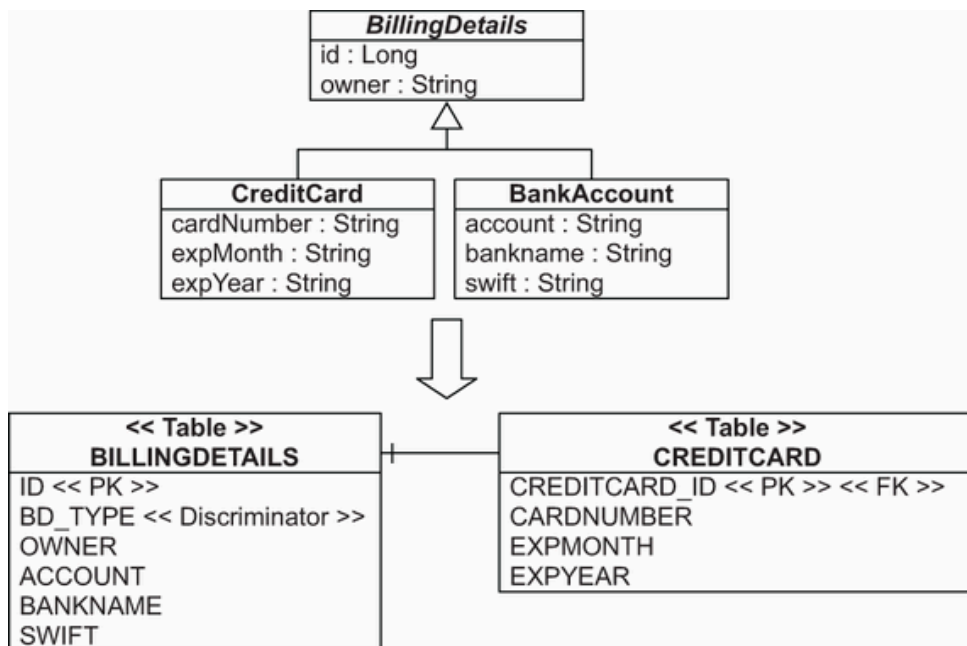


Figure 7.4 Breaking out a subclass to its own secondary table

We'll map the superclass `BillingDetails` with

`InheritanceType.SINGLE_TABLE`, as we did before. Then we'll map the `CreditCard` subclass we want to break out of the single table to a secondary table.

Listing 7.18 Mapping `CreditCard`

```
Path: Ch07/mapping-inheritance-mixed/src/main/java/com/manning
➡ /javapersistence/ch07/model/CreditCard.java
```

```
@Entity
@DiscriminatorValue("CC")
@SecondaryTable(
    name = "CREDITCARD",
    pkJoinColumns = @PrimaryKeyJoinColumn(name = "CREDITCARD_ID")
)
public class CreditCard extends BillingDetails {
    @NotNull
    @Column(table = "CREDITCARD", nullable = false)
    private String cardNumber;

    @Column(table = "CREDITCARD", nullable = false)
    private String expMonth;

    @Column(table = "CREDITCARD", nullable = false)
    private String expYear;
    // . . .
}
```

The `@SecondaryTable` and `@Column` annotations group some properties and tell Hibernate to get them from a secondary table. We map all the properties that we moved into the secondary table with the name of that secondary table. This is done with the `table` parameter of `@Column`, which we haven't shown before. This mapping has many uses, and you'll see it again later in this book. In this example, it separates the `CreditCard` properties from the single table strategy into the `CREDITCARD` table. This would be a viable solution if we wanted to add a new class to extend `BillingDetails`; `Paypal` for example.

The `CREDITCARD_ID` column of this table is also the primary key, and it has a foreign key constraint referencing the `ID` of the single hierarchy table. If we don't specify a primary key join column for the secondary table, the name of the primary key of the single inheritance table is used—in this case, `ID`.

Remember that `InheritanceType.SINGLE_TABLE` enforces all columns of subclasses to be nullable. One of the benefits of this mapping is that we can now declare columns of the `CREDITCARD` table as `NOT NULL`, guaranteeing data integrity.

At runtime, Hibernate executes an outer join to fetch `BillingDetails` and all subclass instances polymorphically:

```
select
    ID, OWNER, ACCOUNT, BANKNAME, SWIFT,
```

```

        EXPMONTH, EXPYEAR, CARDNUMBER,
        BD_TYPE
    from
        BILLINGDETAILS
    left outer join CREDITCARD on ID=CREDITCARD_ID

```

We can also use this trick for other subclasses in the class hierarchy. For an exceptionally wide class hierarchy, the outer join can become a problem. Some database systems (Oracle, for example) limit the number of tables in an outer join operation. For a wide hierarchy, you may want to switch to a different fetching strategy that executes an immediate second SQL select instead of an outer join.

7.6 Inheritance of embeddable classes

An embeddable class is a component of its owning entity, so the normal entity inheritance rules presented in this chapter don't apply. As a Hibernate extension, we can map an embeddable class that inherits some persistent properties from a superclass (or interface). Let's consider two new attributes of an auction item: dimensions and weight.

An item's dimensions are its width, height, and depth, expressed in a given unit and its symbol: for example, inches (") or centimeters (cm). An item's weight also carries a unit of measurement: for example, pounds (lbs) or kilograms (kg). To capture the common attributes (name and symbol) of measurement, we'll define a superclass for `Dimensions` and `Weight` called `Measurement`. The source code that follows can be found in the `mapping-inheritance-embeddable` folder.

Listing 7.19 Mapping the `Measurement` abstract embeddable superclass

```

Path: Ch07/mapping-inheritance-embeddable/src/main/java/com/manning
➡ /javapersistence/ch07/model/Measurement.java

@MappedSuperclass
public abstract class Measurement {
    @NotNull
    private String name;
    @NotNull
    private String symbol;
    // . . .
}

```

We've used the `@MappedSuperclass` annotation on the superclass of the embeddable class we're mapping, just like we would for an entity. Subclasses will inherit the properties of this class as persistent properties.

We'll define the `Dimensions` and `Weight` subclasses as `@Embeddable`. For `Dimensions`, we'll override all the superclass attributes and add a column-name prefix.

Listing 7.20 Mapping the `Dimensions` class

Path: `Ch07/mapping-inheritance-embeddable/src/main/java/com/manning`
➡ `/javapersistence/ch07/model/Dimensions.java`

```
@Embeddable
@AttributeOverride(name = "name",
    column = @Column(name = "DIMENSIONS_NAME"))
@AttributeOverride(name = "symbol",
    column = @Column(name = "DIMENSIONS_SYMBOL"))
public class Dimensions extends Measurement {
    @NotNull
    private BigDecimal depth;
    @NotNull
    private BigDecimal height;
    @NotNull
    private BigDecimal width;
    // . . .
}
```

Without this override, an `Item` embedding both `Dimensions` and `Weight` would map to a table with conflicting column names.

Next is the `Weight` class; its mapping also overrides the column names with a prefix (for uniformity, we avoid the conflict with the previous override).

Listing 7.21 Mapping the `Weight` class

Path: `Ch07/mapping-inheritance-embeddable/src/main/java/com/manning`
➡ `/javapersistence/ch07/model/Weight.java`

```
@Embeddable
@AttributeOverride(name = "name",
    column = @Column(name = "WEIGHT_NAME"))
@AttributeOverride(name = "symbol",
    column = @Column(name = "WEIGHT_SYMBOL"))
public class Weight extends Measurement {
    @NotNull
    @Column(name = "WEIGHT")
    private BigDecimal value;
    // . . .
}
```

The owning entity `Item` defines two regular persistent embedded properties.

Listing 7.22 Mapping the `Item` class

```
Path: Ch07/mapping-inheritance-embeddable/src/main/java/com/manning
➡ /javapersistence/ch07/model/Item.java
```

```
@Entity
public class Item {
    private Dimensions dimensions;
    private Weight weight;
    // . . .
}
```

Figure 7.5 illustrates this mapping.

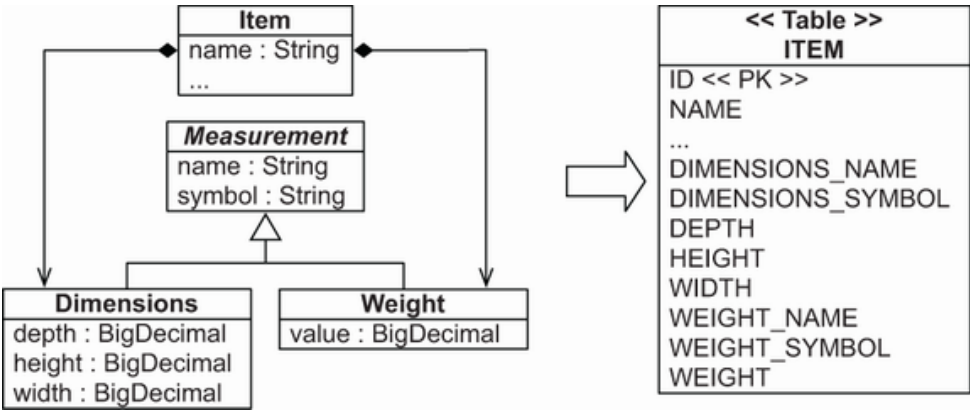


Figure 7.5 Mapping concrete embeddable classes with their inherited properties

Alternatively, we could override the conflicting `Measurement` column names of the embedded properties in the `Item` class, as was demonstrated in section 6.2. However, we prefer to override them once in the `@Embeddable` classes, so consumers of these classes don't have to resolve the conflict.

A pitfall to watch out for is embedding a property of an abstract superclass type (like `Measurement`) in an entity (like `Item`). This can never work; the JPA provider doesn't know how to store and load

`Measurement` instances polymorphically. It doesn't have the information necessary to decide whether the values in the database are `Dimensions` or `Weight` instances because there is no discriminator. This means although we *can* have an `@Embeddable` class inherit some persistent properties from a `@MappedSuperclass`, the reference to an instance isn't polymorphic—it always names a concrete class.

Compare this with the alternative inheritance strategy for embeddable classes examined in the section “Converting properties of components”

(within section 6.3.2), which supported polymorphism but required some custom type-discrimination code.

Next we'll provide some tips on how to choose an appropriate combination of mapping strategies for an application's class hierarchies.

7.7 Choosing a strategy

Which inheritance-mapping strategy you choose will depend on how superclasses are used in the entity hierarchy. You'll have to consider how frequently you query for instances of the superclasses and whether you have associations targeting the superclasses. Another important aspect is the attributes of super- and subtypes: whether subtypes have many additional attributes or only different behavior than their supertypes. Here are some rules of thumb:

- If you don't require polymorphic associations or queries, lean toward table-per-concrete class—in other words, if you never or rarely `select bd from BillingDetails bd`, and you have no class that has an association to `BillingDetails`. An explicit `UNION`-based mapping with `InheritanceType.TABLE_PER_CLASS` should be preferred because (optimized) polymorphic queries and associations will be possible later.
- If you do require polymorphic associations (an association to a superclass, and hence to all classes in the hierarchy with a dynamic resolution of the concrete class at runtime) or queries, and subclasses declare relatively few properties (particularly if the main difference between subclasses is in their behavior), lean toward `InheritanceType.SINGLE_TABLE`. This approach can be chosen if it involves setting a minimal number of columns as nullable. You'll need to convince yourself (and the DBA) that a denormalized schema won't create problems in the long run.
- If you do require polymorphic associations or queries, and subclasses declare many (non-optional) properties (subclasses differ mainly by the data they hold), lean toward `InheritanceType.JOINED`. Alternatively, depending on the width and depth of the inheritance hierarchy and the possible cost of joins versus unions, use `InheritanceType.TABLE_PER_CLASS`. This decision might require the evaluation of SQL execution plans with real data.

By default, choose `InheritanceType.SINGLE_TABLE` only for simple problems. For complex cases, or when a data modeler insists on the importance of `NOT NULL` constraints and normalization overrules you, you should consider the `InheritanceType.JOINED` strategy. At that point, you should ask yourself whether it may not be better to remodel inheritance as delegation in the class model. Complex inheritance is often

best avoided for all sorts of reasons unrelated to persistence or ORM. Hibernate acts as a buffer between the domain and relational models, but that doesn't mean you can ignore persistence concerns completely when designing the classes.

When you start thinking about mixing inheritance strategies, you must remember that implicit polymorphism in Hibernate is smart enough to handle exotic cases. Also, you must consider that you can't put inheritance annotations on interfaces; this isn't standardized in JPA.

For example, suppose you needed to add an interface in the CaveatEmptor application: `ElectronicPaymentOption`. This is a business interface that doesn't have a persistence aspect except that a persistent class such as `CreditCard` will likely implement this interface. No matter how we map the `BillingDetails` hierarchy, Hibernate can answer the query `select o from ElectronicPaymentOption o` correctly. This even works if other classes, which aren't part of the `BillingDetails` hierarchy, are mapped as persistent and implement this interface. Hibernate always knows what tables to query, which instances to construct, and how to return a polymorphic result.

We can apply all mapping strategies to abstract classes. Hibernate won't try to instantiate an abstract class, even if we query or load it.

We mentioned the relationship between `User` and `BillingDetails` several times, and we looked at how it influences the selection of an inheritance-mapping strategy. In the following and last section of this chapter, we'll explore this more advanced topic in detail: polymorphic associations. If you don't have such a relationship in your model right now, you may want to come back to this topic later, when you encounter the issue in your application.

7.8 Polymorphic associations

Polymorphism is a defining feature of object-oriented languages like Java. Support for polymorphic associations and polymorphic queries is a fundamental feature of an ORM solution like Hibernate. Surprisingly, we've managed to get this far without needing to talk much about polymorphism. Refreshingly, there isn't much to say on the topic—polymorphism is so easy to use in Hibernate that we don't need to expend a lot of effort explaining it.

To provide an overview, we'll first consider a *many-to-one* association to a class that may have subclasses and then a *one-to-many* relationship. For both examples, the classes of the domain model are the same; see figure 7.6.

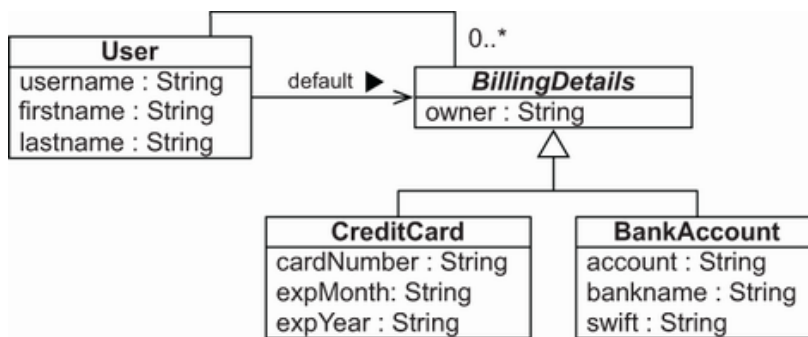


Figure 7.6 A user has either a credit card or a bank account as the default billing details.

7.8.1 Polymorphic many-to-one associations

First, consider the `defaultBilling` property of `User`. It references one particular `BillingDetails` instance, which at runtime can be any concrete instance of that class. The source code that follows can be found in the `mapping-inheritance-manytoone` folder.

We'll map this unidirectional association to the abstract class `BillingDetails` as follows:

Path: `Ch07/mapping-inheritance-manytoone/src/main/java/com/manning`
 ➔ `/javapersistence/ch07/model/User.java`

```

@Entity
@Table(name = "USERS")
public class User {
    @ManyToOne
    private BillingDetails defaultBilling;
    // . . .
}
  
```

The `USERS` table now has the join/foreign-key column `DEFAULTBILLING_ID` representing this relationship. It's a nullable column because a `User` might not have a default billing method assigned. Because `BillingDetails` is abstract, the association must refer to an instance of one of its subclasses—`CreditCard` or `BankAccount`—at runtime.

We don't have to do anything special to enable polymorphic associations in Hibernate. If the target class of an association is mapped with `@Entity` and `@Inheritance`, the association is naturally polymorphic.

The following Spring Data JPA code demonstrates the creation of an association to an instance of the `CreditCard` subclass:

Path: `Ch07/mapping-inheritance-manytoone/src/test/java/com/manning`
 ➔ `/javapersistence/ch07/MappingInheritanceSpringDataJPATest.java`

```

CreditCard creditCard = new CreditCard(
    "John Smith", "123456789", "10", "2030"
);
User john = new User("John Smith");
john.setDefaultBilling(creditCard);
creditCardRepository.save(creditCard);
userRepository.save(john);

```

Now, when we navigate the association in a second unit of work, Hibernate automatically retrieves the `CreditCard` instance:

```

List<User> users = userRepository.findAll();
users.get(0).getDefaultBilling().pay(123);

```

The second line here will invoke the `pay` method from the concrete subclass of `BillingDetails`.

We can handle *one-to-one* associations the same way. What about plural associations, like the collection of `billingDetails` for each `User`? Let's look at that next.

7.8.2 Polymorphic collections

A `User` may have references to many `BillingDetails`, not only a single default (one of the many is the default, but let's ignore that for now). We can map this with a bidirectional *one-to-many* association. The source code that follows can be found in the `mapping-inheritance-onetomany` folder.

```

Path: Ch07/mapping-inheritance-onetomany/src/main/java/com/manning
➡ /javapersistence/ch07/model/User.java

```

```

@Entity
@Table(name = "USERS")
public class User {
    @OneToMany(mappedBy = "user")
    private Set<BillingDetails> billingDetails = new HashSet<>();
    // . . .
}

```

Next, here's the owning side of the relationship (declared with `mappedBy` in the previous mapping). By the *owning side*, we mean that side of the relationship that owns the foreign key in the database, which is `BillingDetails` in this example.

Path: Ch07/mapping-inheritance-onetomany/src/main/java/com/manning
➡ /javapersistence/ch07/model/BillingDetails.java

```
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class BillingDetails {
    @ManyToOne
    private User user;
    // . . .
}
```

So far, there is nothing special about this association mapping. The `BillingDetails` class hierarchy can be mapped with `TABLE_PER_CLASS`, `SINGLE_TABLE`, or a `JOINED` inheritance type. Hibernate is smart enough to use the right SQL queries, with either `JOIN` or `UNION` operators, when loading the collection elements.

There is one limitation, however: the `BillingDetails` class can't be a `@MappedSuperclass`, as discussed in section 7.1. It has to be mapped with `@Entity` and `@Inheritance`.

Summary

- Table-per-concrete-class with implicit polymorphism is the simplest strategy for mapping inheritance hierarchies of entities, and it doesn't support polymorphic associations very well.
- Different columns from different tables share exactly the same semantics, making schema evolution more complex.
- This table-per-concrete-class approach is recommended for the top level of the class hierarchy only, where polymorphism isn't usually required and when modification of the superclass in the future is unlikely.
- The table-per-concrete-class-with-unions strategy is optional, and JPA implementations may not support it, but it does handle polymorphic associations.
- The table-per-class-hierarchy strategy is a winner in terms of both performance and simplicity. Ad hoc reporting is possible without complex joins or unions, and schema evolution is straightforward.
- The one major problem with the single-table strategy is data integrity, because we must declare some columns as nullable. Another concern is normalization: this strategy creates functional dependencies between non-key columns, violating the third normal form.

- The table-per-subclass-with-joins strategy has as its primary advantage the fact that it normalizes the SQL schema, making schema evolution and integrity constraint definition straightforward. The disadvantages are that it's more difficult to implement by hand, and performance can be unacceptable for complex class hierarchies.